

OOP: Interfaces

Visual C# How to Program, 6/e

Case Study: Creating and Using Interfaces

- ▶ **Interfaces** specify the **public services** (methods and properties) that classes must implement
- ▶ Interfaces vs. abstract classes w.r.t default implementations
 - **Interfaces** provide **no** default implementations
 - **Abstract classes** may provide **some** default implementations
 - If **no default implementations can/are defined** – do *not* use an abstract class, **use an interface** instead
- ▶ Interfaces are used to “**bring together**” or **relate to each other** disparate objects that relate to one another only through the interface
 - I.e., provide uniform set of methods and properties for disparate objects
 - E.g.: A **person** and a **tree** are **disparate objects**
 - Interface can define **age** and **name** for these disparate objects
 - Enables **polymorphic** processing of **age** and **name** for person & tree objects

Case Study: Creating and Using Interfaces (Cont.)

- ▶ Interfaces are defined using keyword **interface**
- ▶ Use inheritance notation to specify that a class implements an interface
ClassName : InterfaceName
- ▶ Classes may implement more than one interface:
e.g.: *ClassName : InterfaceName1, InterfaceName2*
Can also have:
ClassName : ClassName1, InterfaceName1, InterfaceName2 (object list must precedes interface list)
- ▶ A class that implement an interface, must provide implementations for every method and property in the interface definition
- ▶ Example: interface **IAge** that returns information about an object's age
 - Can be used by classes for people, cars, trees (all have an age)

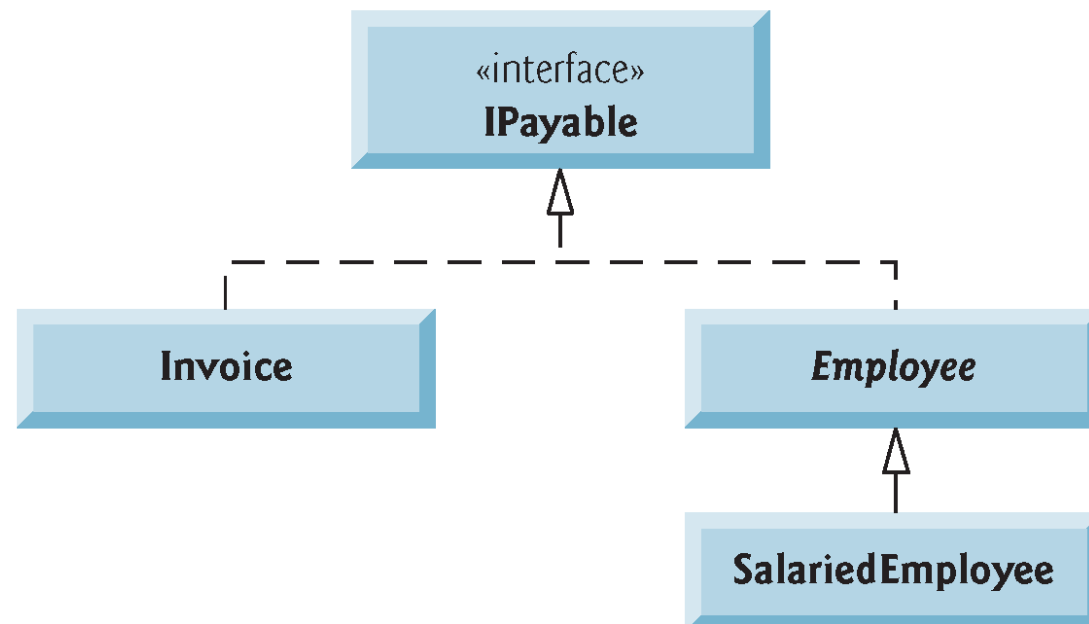
Developing an IPayable Hierarchy

- To build an app that can determine payments for employees and invoices alike, we first create an interface named `IPayable`.
- Interface `IPayable` contains method `GetPaymentAmount` that returns a `decimal` amount to be paid for an object of any class that implements the interface.

Developing an IPayable Hierarchy (Cont.)

UML Diagram Containing an Interface

- ▶ The UML class diagram shows the interface and class hierarchy used in our accounts-payable app.



```
1  // Fig. 12.11: IPayable.cs
2  // IPayable interface declaration.
3  public interface IPayable
4  {
5      decimal GetPaymentAmount(); // calculate payment; no implementation
6  }
```

```
1  // Fig. 12.12: Invoice.cs
2  // Invoice class implements IPayable.
3  using System;
4
5  public class Invoice : IPayable
6  {
7      public string PartNumber { get; }
8      public string PartDescription { get; }
9      private int quantity;
10     private decimal pricePerItem;
11
12     // four-parameter constructor
13     public Invoice(string partNumber, string partDescription, int quantity,
14         decimal pricePerItem)
15     {
16         PartNumber = partNumber;
17         PartDescription = partDescription;
18         Quantity = quantity; // validate quantity
19         PricePerItem = pricePerItem; // validate price per item
20     }
21
```

```
22 // property that gets and sets the quantity on the invoice
23 public int Quantity
24 {
25     get
26     {
27         return quantity;
28     }
29     set
30     {
31         if (value < 0) // validation
32         {
33             throw new ArgumentOutOfRangeException(nameof(value),
34                 value, $"{nameof(Quantity)} must be >= 0");
35         }
36
37         quantity = value;
38     }
39 }
40
```

```
41 // property that gets and sets the price per item
42 public decimal PricePerItem
43 {
44     get
45     {
46         return pricePerItem;
47     }
48     set
49     {
50         if (value < 0) // validation
51         {
52             throw new ArgumentOutOfRangeException(nameof(value),
53                 value, $"{nameof(PricePerItem)} must be >= 0");
54         }
55
56         pricePerItem = value;
57     }
58 }
```

```
59
60 // return string representation of Invoice object
61 public override string ToString() =>
62     $"invoice:\npart number: {PartNumber} ({PartDescription})\n" +
63     $"quantity: {Quantity}\nprice per item: {PricePerItem:C}";
64
65 // method required to carry out contract with interface IPayable
66 public decimal GetPaymentAmount() => Quantity * PricePerItem;
67 }
```

Modifying Class Employee to Implement Interface IPayable

- ▶ GetPaymentAmount simply calls Employee's abstract method Earnings.
- ▶ At execution time, when GetPaymentAmount is called on an object of an Employee derived class, GetPaymentAmount calls that class's concrete Earnings method, which knows how to calculate earnings for objects of that derived-class type.

```
1  // Fig. 12.14: PayableInterfaceTest.cs
2  // Tests interface IPayable with disparate classes.
3  using System;
4  using System.Collections.Generic;
5
6  class PayableInterfaceTest
7  {
8      static void Main()
9      {
10         // create a List<IPayable> and initialize it with four
11         // objects of classes that implement interface IPayable
12         var payableObjects = new List<IPayable>() {
13             new Invoice("01234", "seat", 2, 375.00M),
14             new Invoice("56789", "tire", 4, 79.95M),
15             new SalariedEmployee("John", "Smith", "111-11-1111", 800.00M),
16             new SalariedEmployee("Lisa", "Barnes", "888-88-8888", 1200.00M)};
17
```

```
18 Console.WriteLine(  
19     "Invoices and Employees processed polymorphically:\n");  
20  
21 // generically process each element in payableObjects  
22 foreach (var payable in payableObjects)  
23 {  
24     // output payable and its appropriate payment amount  
25     Console.WriteLine($"{payable}");  
26     Console.WriteLine(  
27         $"payment due: {payable.GetPaymentAmount():C}\n");  
28     }  
29 }  
30 }
```

Invoices and Employees processed polymorphically:

invoice:

part number: 01234 (seat)

quantity: 2

price per item: \$375.00

payment due: \$750.00

invoice:

part number: 56789 (tire)

quantity: 4

price per item: \$79.95

payment due: \$319.80

salaried employee: John Smith

social security number: 111-11-1111

weekly salary: \$800.00

payment due: \$800.00

salaried employee: Lisa Barnes

social security number: 888-88-8888

weekly salary: \$1,200.00

payment due: \$1,200.00

Common Interfaces of the .NET Framework Class Library

- ▶ Figure overviews several commonly used Framework Class Library interfaces.

Interface	Description
IComparable	C# contains several comparison operators (e.g., <, <=, >, >=, ==, !=) that allow you to compare simple-type values. Section 10.13 showed that you can overload these operators for your own types. Interface <code>IComparable</code> can be used to allow objects of a class that implements the interface to be compared to one another. The interface contains one method, <code>CompareTo</code>, which compares the object that calls the method to the object passed as an argument. Classes must implement <code>CompareTo</code> to return a value indicating whether the object on which it's invoked is less than (negative integer return value), equal to (0 return value) or greater than (positive integer return value) the object passed as an argument, using any criteria you specify. For example, if class <code>Employee</code> implements <code>IComparable</code>, its <code>CompareTo</code> method could compare <code>Employee</code> objects by their earnings amounts. Interface <code>IComparable</code> is commonly used for ordering objects in a collection such as an array. We use <code>IComparable</code> in Chapter 20, Generics, and Chapter 21, Generic Collections; Functional Programming with LINQ/PLINQ.

Interface	Description
IComponent	Implemented by any class that represents a component, including Graphical User Interface (GUI) controls (such as buttons or labels). Interface IComponent defines the behaviors that components must implement. We discuss IComponent and many GUI controls that implement this interface in Chapter 14, Graphical User Interfaces with Windows Forms: Part 1, and Chapter 15, Graphical User Interfaces with Windows Forms: Part 2.

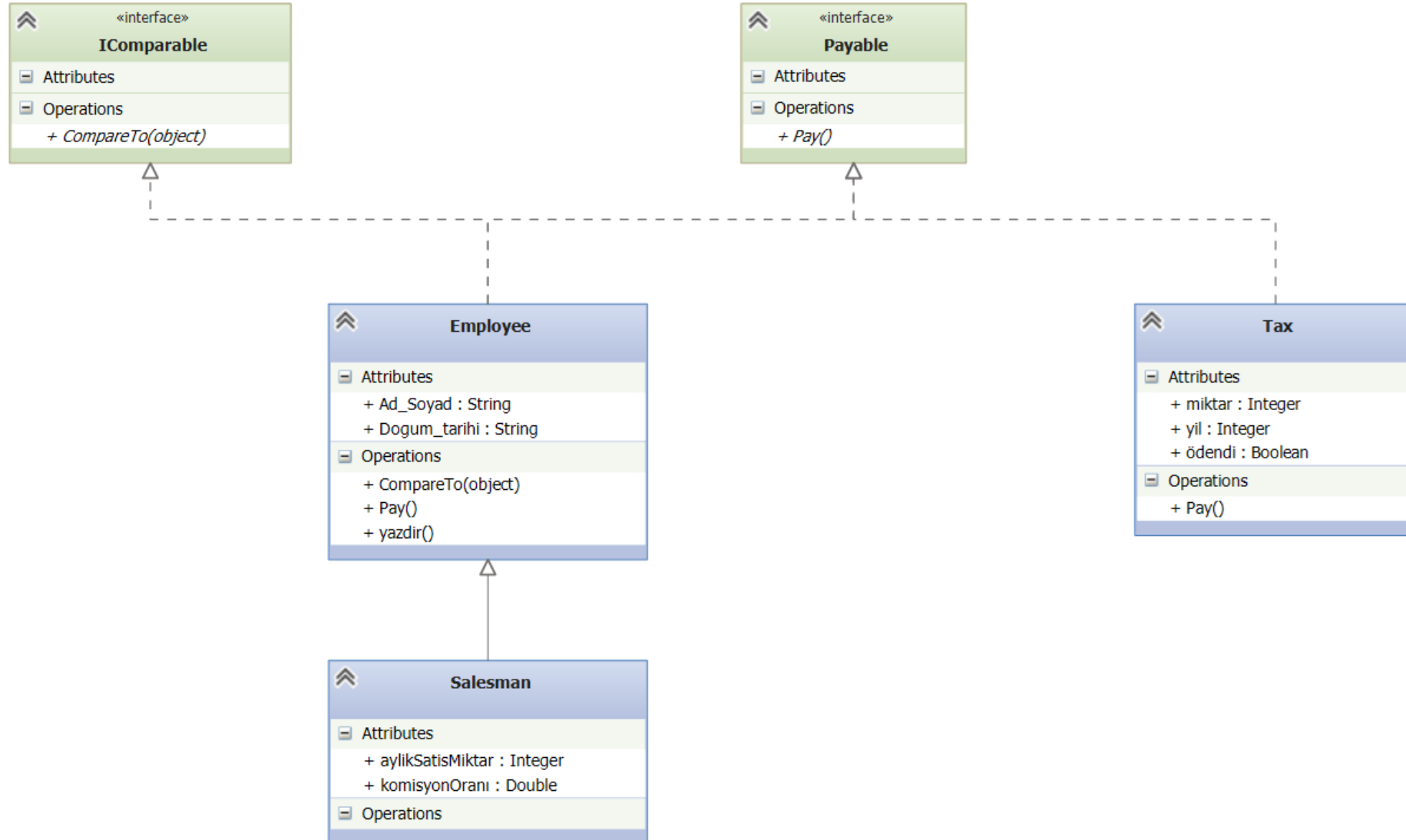
Interface	Description
IDisposable	Implemented by classes that must provide an explicit mechanism for <i>releasing</i> resources. Some resources can be used by only one program at a time. In addition, some resources, such as files on disk, are unmanaged resources that, unlike memory, cannot be released by the garbage collector. Classes that implement interface <code>IDisposable</code> provide a <code>Dispose</code> method that can be called to explicitly release resources that are explicitly associated with an object. We discuss <code>IDisposable</code> briefly in Chapter 13, <i>Exception Handling: A Deeper Look</i>. You can learn more about this interface at http://msdn.microsoft.com/library/system.idisposable. The MSDN article <i>Implementing a Dispose Method</i> at http://msdn.microsoft.com/library/fs2xkftw discusses the proper implementation of this interface in your classes.

Interface	Description
IEnumerator	Used for iterating through the elements of a <i>collection</i> (such as an array or a <code>List</code>) one element at a time—the <code>foreach</code> statement uses an <code>IEnumerator</code> object to iterate through elements. Interface <code>IEnumerator</code> contains method <code>MoveNext</code> to move to the next element in a collection, method <code>Reset</code> to move to the position before the first element and property <code>Current</code> to return the object at the current location. We use <code>IEnumerator</code> in Chapter 21. All <code>IEnumerable</code> objects (Chapter 9) provide a <code>GetEnumerator</code> method that returns an <code>IEnumerator</code> object.

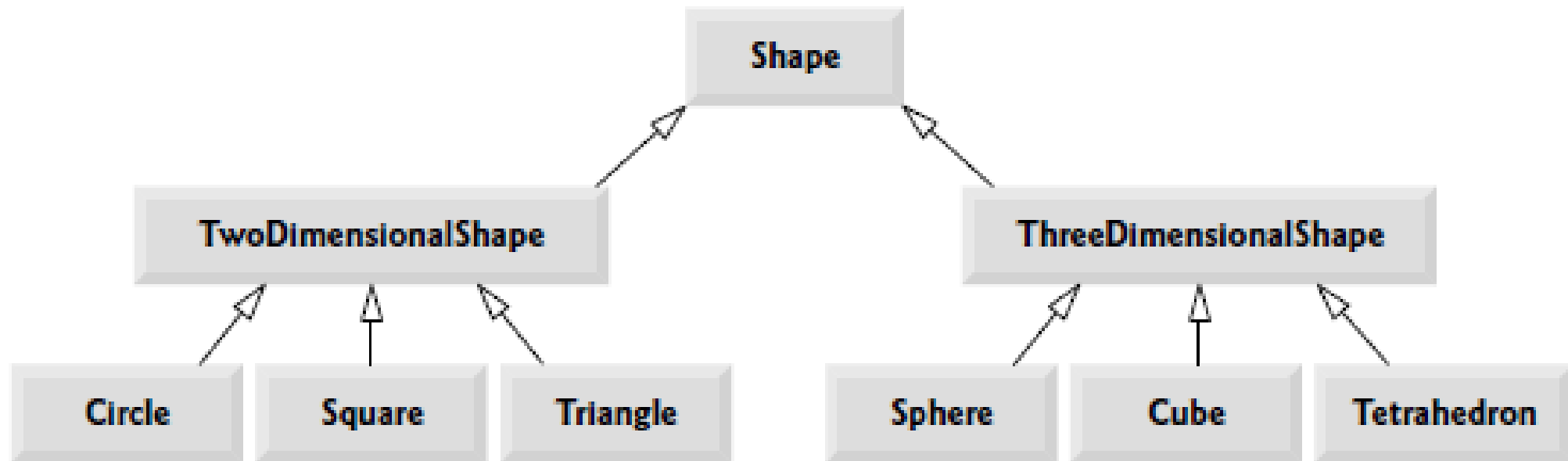
Örnek 1 – Çalışan ve Vergi

- Ad, soyad ve aylık maaş özelliklerine sahip bir *Employee* sınıfını ve yılı, miktarı ve ödendi bilgisi özelliklerine sahip *Tax* sınıfı tarafından uygulanacak *Payable* arayüzü oluşturalım ve bu arayüz için de *Pay* isimli bir metot tanımlayın. Bu metot sınıflar tarafında uygun şekilde doldurulacaktır. (Örneğin çalışanda maaş ödemesi, vergide vergi ödemesi.)
- Ayrıca *Employee* sınıfından türetilmiş *Salesman sınıfı* örnekleri aylık maaşlarının yanında satışlarının üzerinden komisyon almaktadırlar.
 - *Employee* sınıfına adını ve yıllık kazancını yazdıran bir metot ekleyelim.
 - *Salesman* sınıfında ilave olarak aylık satış miktarı (para cinsinden) ve komisyon oranı özellikleri olsun.
- Ayrıca *Employee* sınıfının *Comparable* arayüzünün uygulayarak, kendi aralarından doğum tarihlerine göre sıralanabilir olmasını sağlayın. Böylece bir dizi (Array) içine yerleştirdiğiniz *Employee* örneklerini sıralayabileceksiniz. Not: ArrayList içindeki örnekleri sıralanması için *Comparer* arayüzü kullanılır.
- Tüm özellikleri test edebilecek şekilde Main() metodunu düzenleyiniz.

Örnek 1 – Çalışan ve Vergi



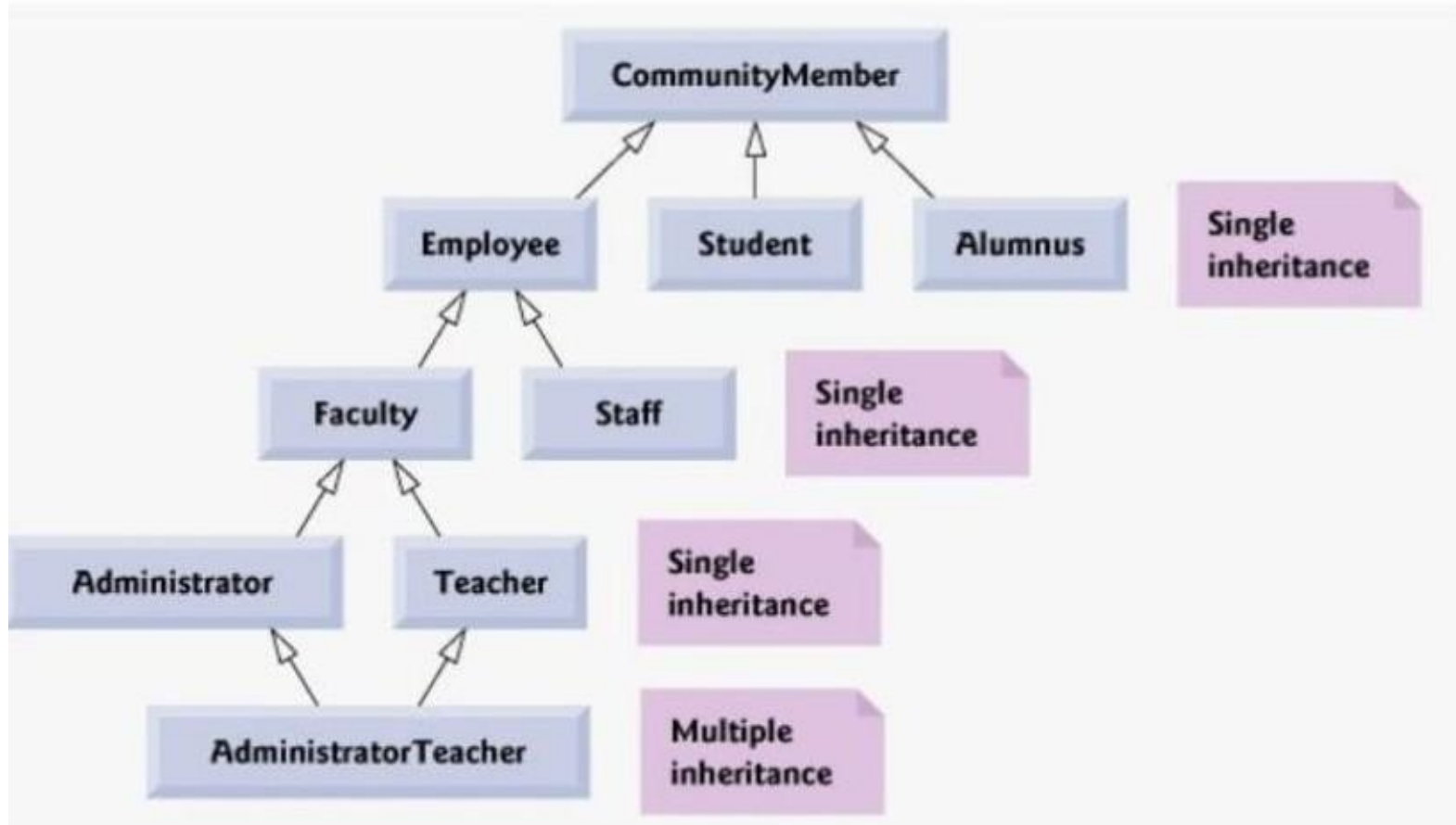
Örnek 2- Şekiller



Örnek 2- Şekiller

- *TwoDimensionalShape* sınıfı ve *ThreeDimensionalShape* sınıfı tarafında kalıtılacak *Shape* soyut sınıfının oluşturulmalıdır.
- *Shape* soyut sınıfında
 - ilgili nesnenin ismini (Circle, Square vb..) döndürecek şekilde uygulanmış bir toString() metodu olmalıdır.
- Her bir alt sınıf için ilgili getArea() metotlar özelleştirilmelidir.
- *TwoDimensionalShape* sınıfında
 - getArea() metodu
- *ThreeDimensionalShape* sınıfında
 - getArea() ve getVolume() metotları olmalıdır.
- Ayrıca *Shape* sınıfı IComparable arayüzünü uygulayarak, dizi içindeki sınıf örneklerinin alanlarına göre sıralanması sağlanmalıdır.
- Bu özellikleri sağlayacak şekilde sınıf hiyerarşisi ve sınıf içleri doldurulmalı, Main() metodu bun göre dizayn edilmelidir.

Örnek 3 Üniversite Sorunsalı



Bilindiği üzere C# 'da çoklu kalıtıma izin verilmemektedir. Yanda ki UML diyagramında ise çoklu kalıtım gerektiren bir durum söz konusudur. Bu durumda ki öneriniz nedir?